Grothendieck Topos

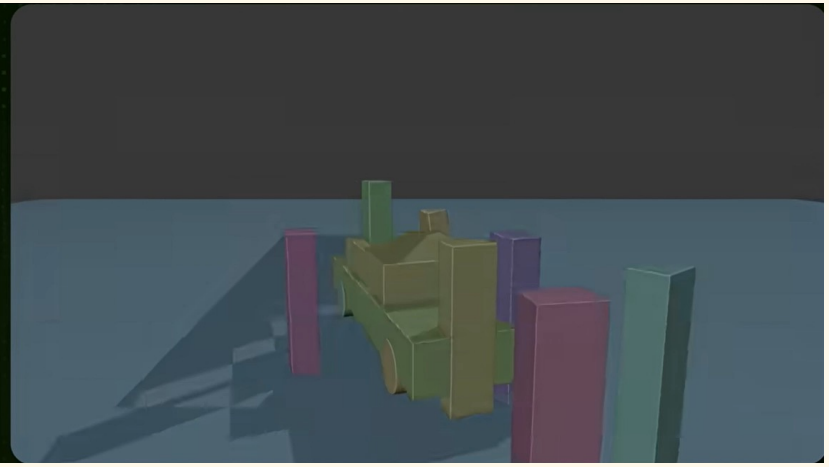
The (only?) thing you should learn in AI era



YANG YUAN

We now live in a crazy era







SUPERMAN

3000 BC → 2020s

twenty styles, from the rock face
to the jacket patch

Twenty ways of drawing one man who never was.

EVERY PLATE IS DRAWN LIVE IN JAVASCRIPT · NO IMAGES



01 PETROGLYPH
Pecked sandstone, c. 3000 BC



02 TOMB PAINTING
Thebes, c. 1300 BC



03 MOSAIC
Pompeii, c. 79 AD



04 STAINED GLASS
Gothic lancet, c. 1220



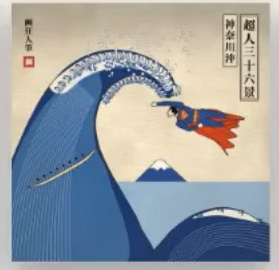
05 ILLUMINATED INITIAL
Book of hours, c. 1410



06 PROPORTION STUDY
After Leonardo, c. 1490



07 SILHOUETTE
Cut paper, c. 1790



08 UKIYO-E
After Hokusai, 1831



09 SAMPLER
Cross-stitch, 1830



10 STRONGMAN BILL
Letterpress, 1870



11 CONSTRUCTIVISM
Montage, 1960



12 ART DECO
Streamline poster, 1930



13 GOLDEN AGE
Newsstand, 1938



14 NEON
Diner sign, 1930s



15 SILKSCREEN
After Warhol, 1960s



16 LINE PRINTER
ASCII art, 1970s



17 HANDHELD
Four shades of green, 1989



18 LOW POLY
Polygon era, 1999



19 STENCIL
Spray paint, 2008



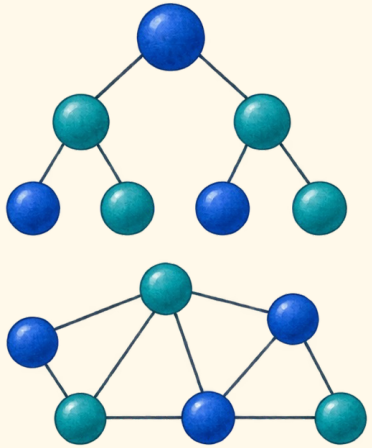
20 PATCH
Embroidered, 2020

Students start asking new questions

A QUESTION I HEAR OFTEN

> Do I still need to learn algorithms?
> Do I still need to learn coding?
> What should I learn so that AI will not replace me?

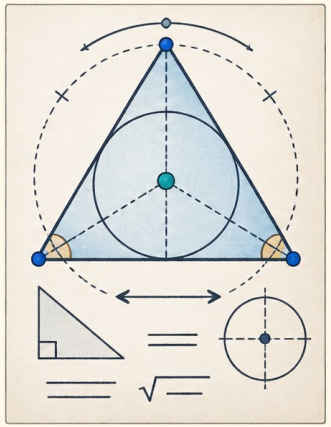
Underlying concern:
What will remain worth learning?



data structures



coding



mathematics

MY ANSWER

Grothendieck Topos

What Truly Defines Future AI?

THE FUTURES WE IMAGINE

01 | Intelligent Robots

02 | 10,000× Faster Science

03 | People Never Die

04 | Infinite Energy & Resources

05 | Build City on Mars Or Another Galaxy

Extraordinary outcomes — but not a definition.

THE ANALOGY

Computers compute.

What does AI fundamentally do?

AI Creates Worlds

But what does it even mean?

DEFINE	What is a world?
CREATE	How do we create one?
UNDERSTAND	How do we understand it?
CERTIFY	How do we know it is sound?

Define A World

Real & living
physical · biological

Simulated
cartoon · game

Formal
mathematics · logic · software

Domain-specific
science · economics



Can **one mathematical language** represent them all?

Create Different Worlds



GAMES

Create an interactive game world



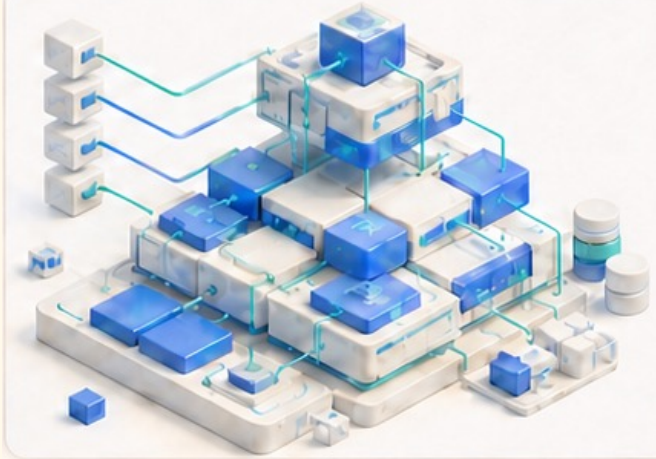
TV SHOWS

Generate 10,000 hours shows automatically



NOVELS

Write a 10,000-page novel



SOFTWARES

Write 1,000,000,000 lines of code with elegant structure

Is there **a systematic way** to construct them?

Understand A World

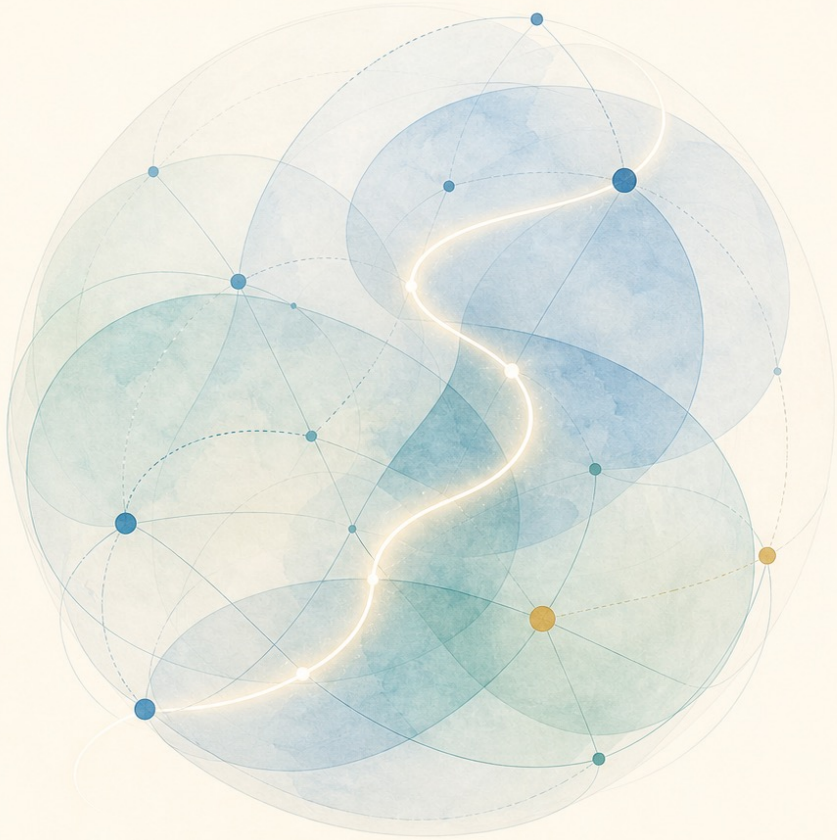
Concepts | Structures

Connections | Behaviors

Proof is the route from local facts to global understanding

When mathematicians prove, "What they really want is not answers — What they want is understanding."

— William Thurston, On Proof and Progress in Mathematics (1994)



Can AI **generate proofs at the scale** of an entire world?

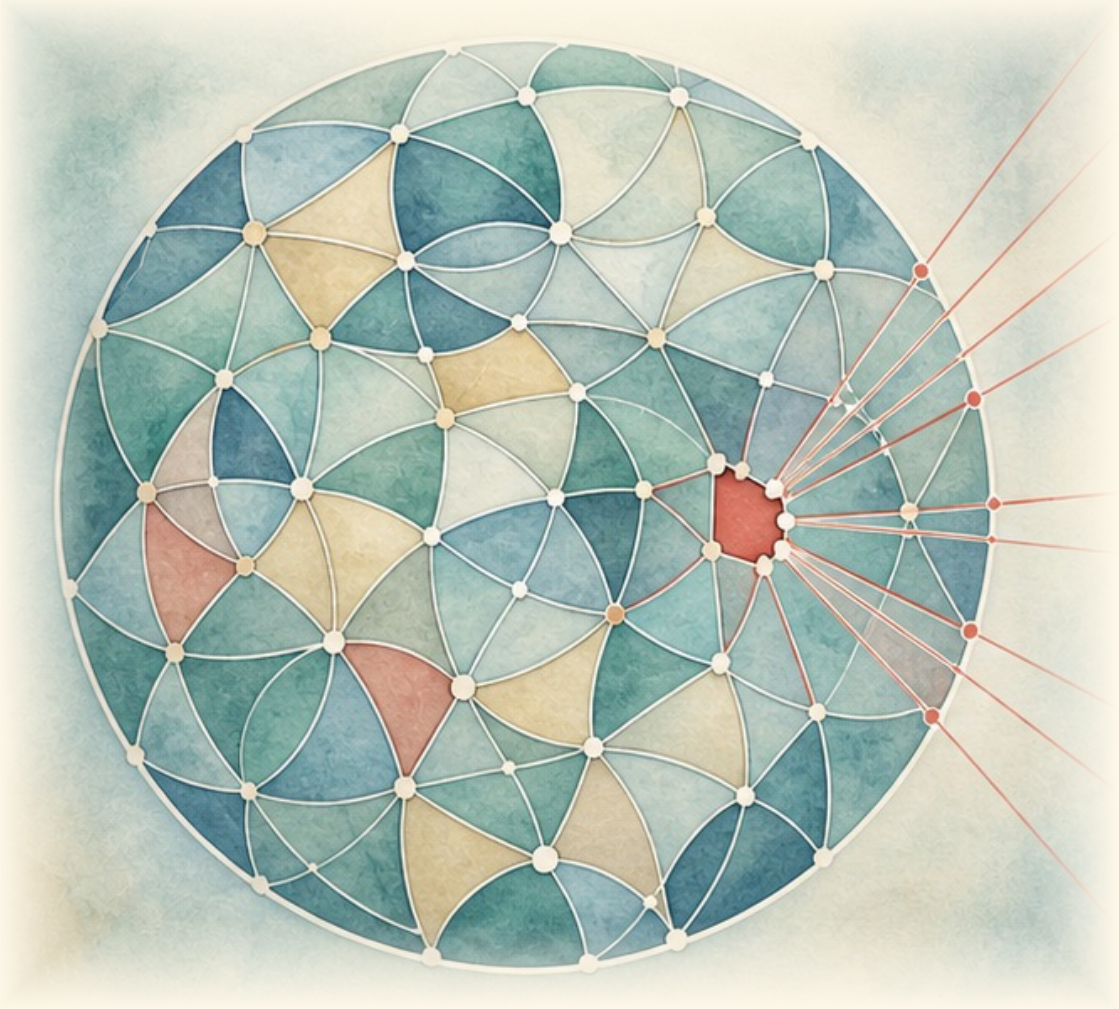
Certify A World

Attack is asymmetric

If a gap exists, a stronger model (like Mythos) will eventually find it

ADVERSARY focuses 10x compute on one weak point

CREATOR must certify every local interface



Can **local certificates guarantee** global construction?

Grothendieck Topos

- Defines worlds
- Can create/understand/certify worlds

What truly defines future AI?

AI creates Grothendieck Topos



Why Shall We Care?

A Theory About Organization/Gluing



《Abstract Elephant》 · By Annemarie Laffont



AI Generated

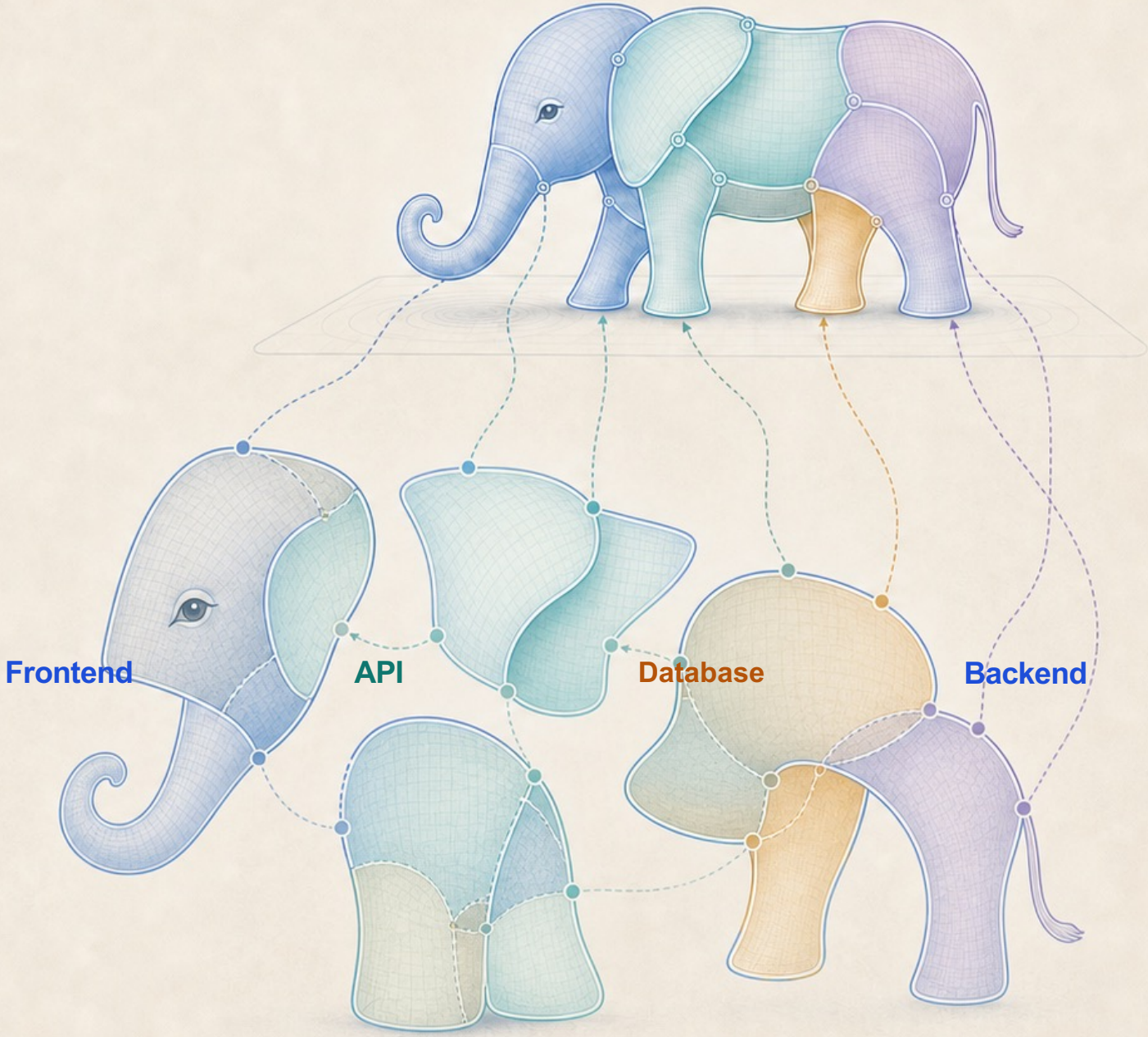
Weird, fragmented, Strange, Incoherent

Natural, living, beautiful, Coherent

Differentiation makes local task spaces checkable

A decomposition acts on a **task space**.

1. **Local task objects**
   typed goals + local contexts
2. **Integration rule**
   handoffs + sheaf-compatible overlaps
3. **Soundness certificate**
   local solutions imply parent validity



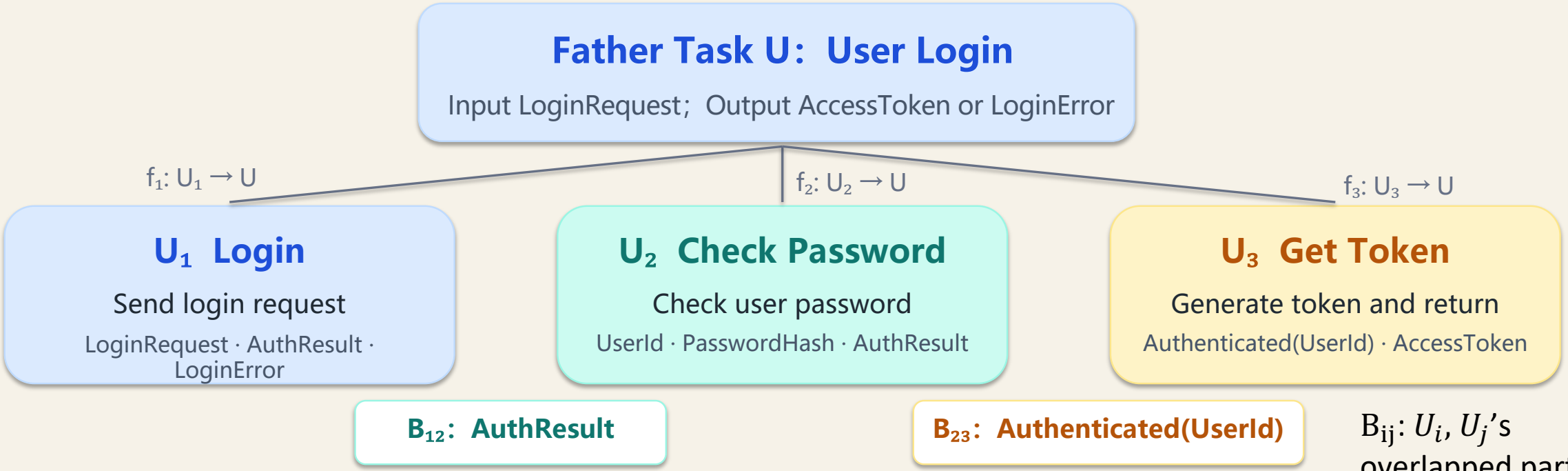
Structure is the solution!

Every decomposition step generates pure monadic plan, only leaf node may have side effects



Understand the structure is equally important as training models

Valid decomposition (cover)



B_{ij}: U_i, U_j's overlapped parts

Information Completeness $e = [f_1, f_2, f_3] : \coprod_{i=1}^{3} U_i \twoheadrightarrow U$

Boundary Consistency $U_{ij} = U_i \times_U U_j, \qquad b_i \pi_i = b_j \pi_j : U_{ij} \to B_{ij}$

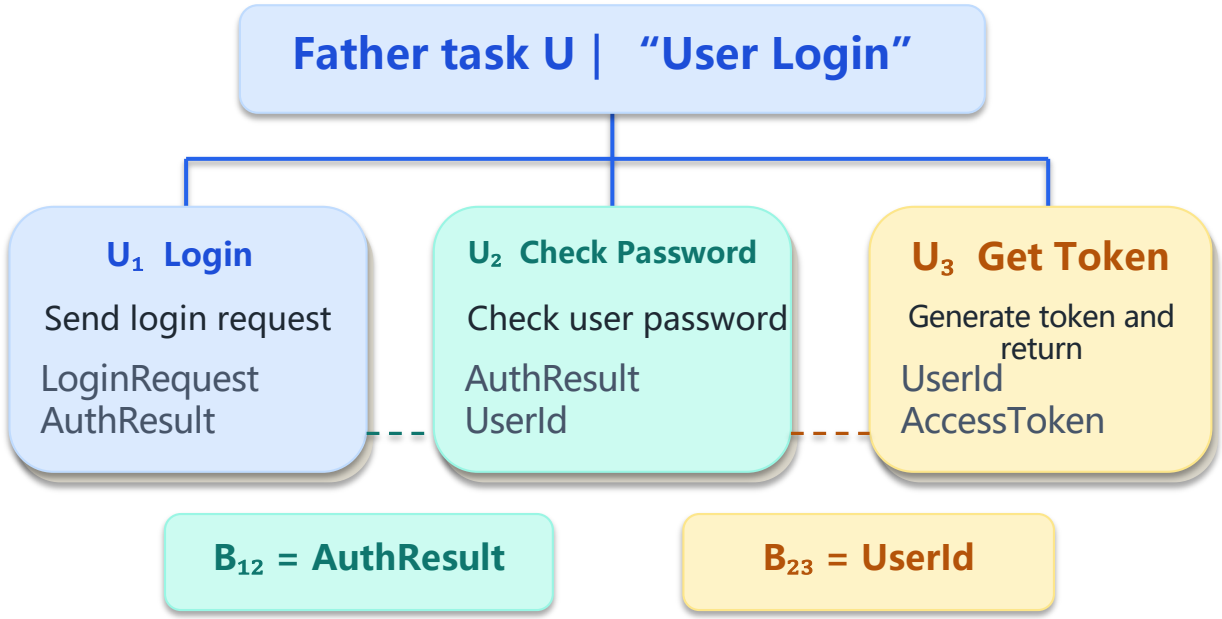
A cover does not give you full code
It only tells decomposition is good

Define category

Assumption	Reason	Intuition in software
Small	We need small site to construct Grothendieck topos	Small = types, interfaces, and contracts form a set rather than a proper class. A small description language can still generate a large universe of tasks.
Regular	Regular epimorphisms are stable under pullback, and each is the coequalizer of its kernel pair	Regular epimorphism = complete coverage after merging; pullback = restriction to a specific subproject. A general decomposition therefore remains valid when restricted to a specific context. Kernel pair means local pieces pointing to the same father; coequalizer means find these pieces, reconstruct the global piece. **Assemble with confidence.**
Extensive	Finite coproducts are compatible with pullbacks; the full kernel pair decomposes into all pairwise overlaps	Coproduct = a disjoint union with module labels; pairwise overlap = the interface actually shared by two modules; extensivity = global conflicts can be reduced to pairwise checks. Thus the full overlap consists entirely of pairwise module overlaps, with no hidden components. **Decompose with confidence.**
Cartesian Closed	Cartesian closure means that products and exponential objects exist.	Product = a record or a combined context; exponential object = a pure function interface. Copying refers to type descriptors, not actual databases. **This allows complex structures and functions to be represented.**

Define Atomic Cover

Cover in code $\mathcal{U} = \{ f_i : U_i \to U \}$



Atomic Cover: two checks

① Information Completeness $[f_i] : \coprod_i U_i \to U$ is a regular epimorphism.

In code: information about requests, validation results, tokens, errors, and so on is not lost during decomposition.

② Boundary Consistency $U_{ij} = U_i \times_u U_j, \quad b_i p_i = b_j p_j$

In code: the interface and authentication layers use the same definition of AuthResult; the authentication and token layers use the same definition of UserId.

K_0 = all basic decompositions approved in advance that pass the checks above.

"Atomic" means a basic rule for generating other valid decompositions, rather than something that cannot be decomposed further.

Defining Operational Covers K: All Valid Decompositions Constructible from K_0

1 Identity Cover

$$\{ id_u : U \to U \} \in K(U)$$

In Software

Leave User Login undecomposed for now and assign the entire task to one agent or developer.

Purpose: allow a task to become a leaf task, so recursive decomposition can stop at an appropriate point.

2 Context Restriction (Base Change)

$$\{ U_i \times_u V \to V \} \in K(V)$$

In Software: V = Login with an SMS code

Interface layer → SMS login interface
Authentication layer → Verification code validation
Token layer → Mobile tokens

Purpose: restrict a general decomposition to a specific platform, customer, or feature context; the boundary declarations are pulled back as well.

3 Recursive Refinement

$$V_{ij} \to U_i \to U$$

In Software: Further Decompose U_2, Authentication

Look up the userValidate credentialsCheck whether the account is locked

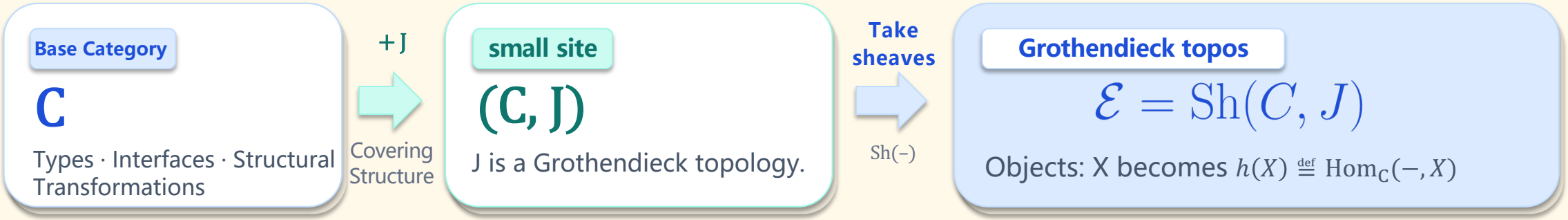
Purpose: local tasks can be decomposed further. Each step is valid, and their composition still covers the original parent task U.

K(U) = all valid type covers obtained from K_0 by applying the three operations above finitely many times.

Identity cover ↔ Maximal sieve Base change ↔ Stability under base change Recursive refinement ↔ Locality

Therefore, K can generate a Grothendieck topology J.

Defining a Grothendieck Topos



Def A category equivalent to the category of sheaves on some small site is called a Grothendieck topos.

Therefore: C small + J a Grothendieck topology ⇒ Sh(C, J) is a Grothendieck topos.

Why Move into $\mathcal{E}$?

$\mathcal{E}$ is the mathematical universe for tasks: it provides both **local geometry and internal logic**

Define Free monad: make plan checkable and composable

Normal morphism $h(I) \longrightarrow h(O)$

Only keep the output

Kleisli plan $p : h(I) \longrightarrow M(h(O))$

Keep the monad

External realization

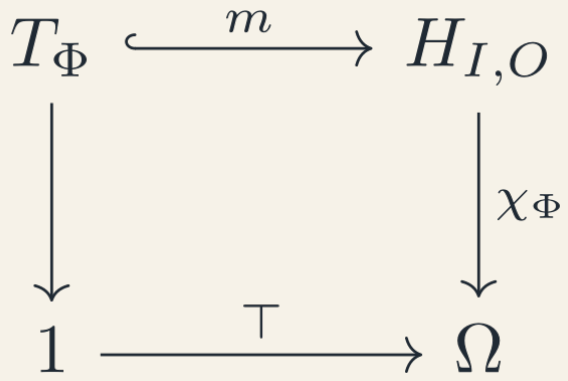
Database, external interactions

Topos decides where the plan lives
Free monad decides how to represent and compose plans

Define subsheaf

Covert all tasks into objects in topos!

$$H_{I,O} = M(h(O))^{h(I)}, \qquad \mathrm{Hom}_{\mathcal{E}}(1, H_{I,O}) \cong \mathrm{Hom}_{\mathcal{E}}(h(I), M(h(O)))$$



$$\chi_\Phi : H_{I,O} \longrightarrow \Omega$$

Φ is formalized specification

$$T_\Phi = \{p \in H_{I,O} \mid \chi_\Phi(p) = \top\}$$

Task is not only an output, but a valid subspace from the whole task space
It is like leader gives you a mission, you shall understand the spirit
Spirit is geometric, but final plan is logical

Defining Gluing: A Projective Limit

T_{v1}
Valid child plan p_1

T_{v2}
Valid child plan p_2

T_{vn}
Valid child plan p_n

H_v
Candidate parent plan p

$$L_{v,d} = \varprojlim F_{v,d} = \{(p_1, \ldots, p_n; p) \mid \text{all equations in } F_{v,d} \text{ hold}\}$$

A point $\ell \in L_{v,d}$

1 Each child plan is locally valid

2 All interfaces and contracts are compatible

3 p is the specified monadic composition

$p \in T_v$ has not yet been established!

Can these local plans coexist as a compatible workflow assembled according to the specified rules?

Main Theorem: Gluing Level by Level Yields the Final Answer

Each Internal Node

First obtain a candidate parent plan from the limit

$$a_{v,d} : L_{v,d} \longrightarrow H_v$$

Certified gluing: every compatible candidate parent plan lies in the parent task space.

Entire Finite Decomposition Tree

Valid Subtasks

$p_\ell \in T_\ell$

Compatibility Holds

$\ell_v \in L_{v,d}$

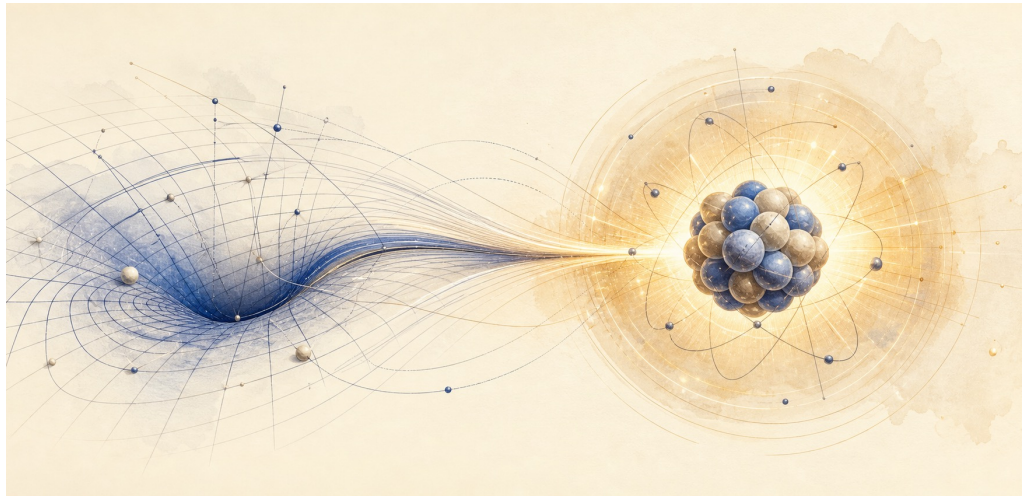
Certified Gluing

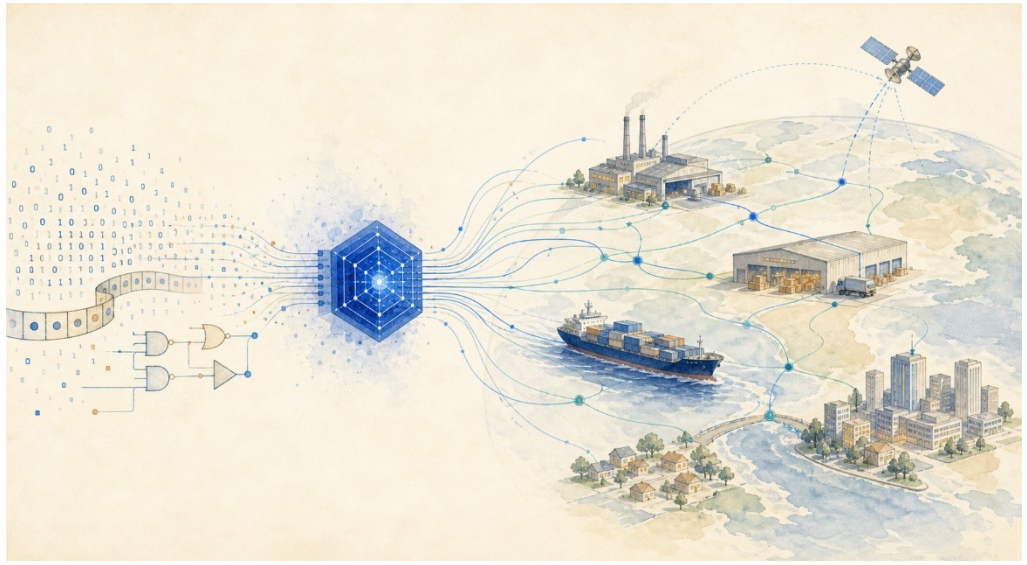
$L_{v,d} \to T_v$

Conclusion $\mathrm{compile}_D(v) \in T_v \implies p_D \in T_\Phi$ at the root

The final compiled global plan satisfies the original specification Φ, relative to the stated external assumptions.

$$E = mc^2$$





INFORMATION + COMPUTATION



GROTHENDIECK TOPOS
ONE THOUGHT, ONE WORLD

Lost Sheep at the Crossroads

A neighbor lost a sheep
everyone went searching
With too many paths, they failed

A story from 列子
about goals, paths, and choices



Lost sheep is not really about sheep

When there are too many paths
The goal is often the first thing we lose

- **Unclear goal**
- **Too many paths**
- **Insufficient judgment**

In the AI era, we face a road network that keeps growing



Big Picture

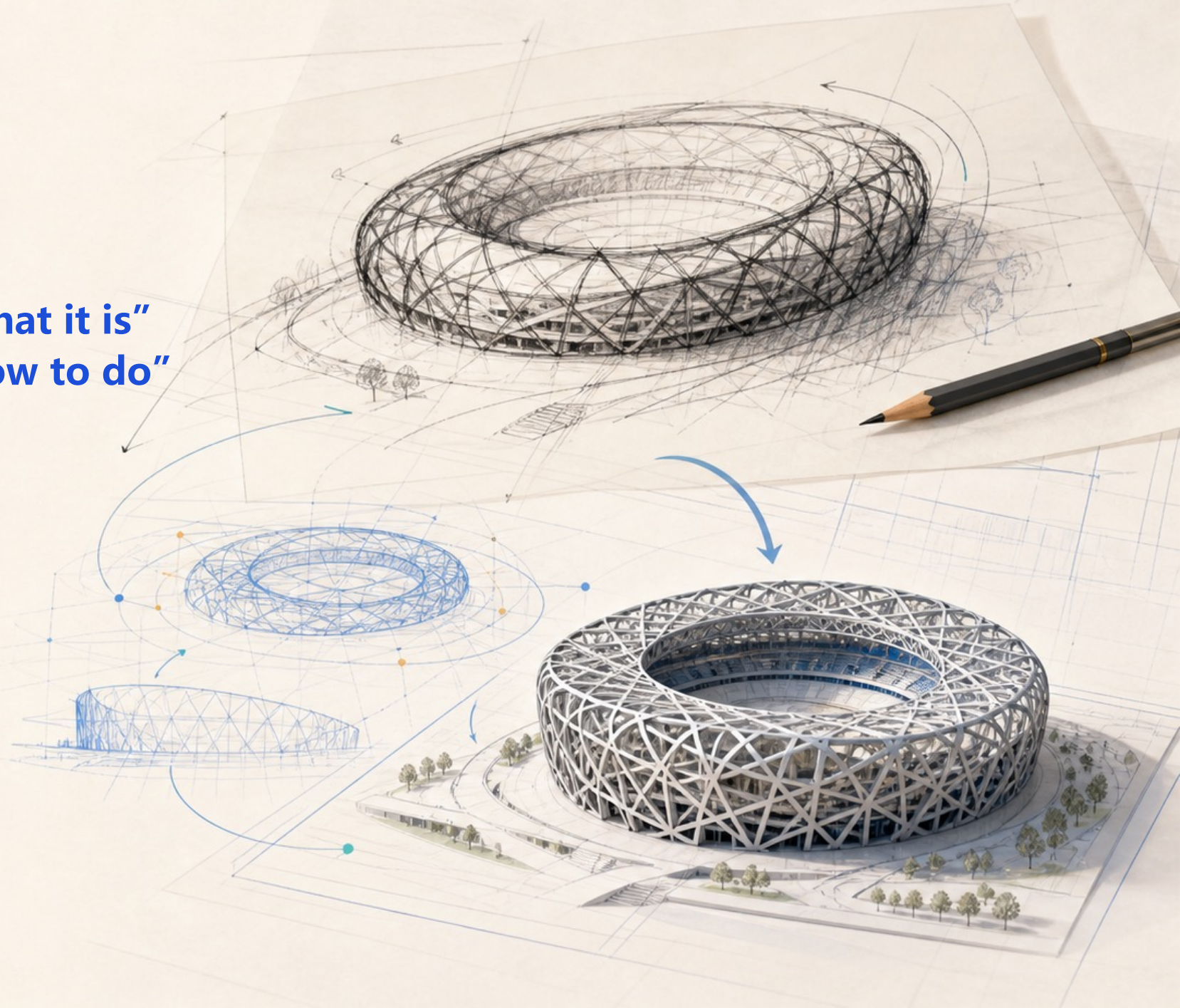
治大国，如烹小鲜

Grasp the whole first
Details stay in order



Architect's vision

Master architect decides “what it is”
Engineering team solves “how to do”



As tools grow stronger, human capabilities move upward

With a combine harvester the sickle is no longer central

Driving a combine harvester raises productivity by dozens of times



The AI era is the same

A system is a small world with rules

A world contains objects and relationships among them

What is in the world?

people objects events data rules

What is between them?

Relationships order feedback boundaries



First capability: imagine a world



Imagine a world you want to create

Second capability: tell AI your world

Having a world in mind does not mean you can hand it clearly to AI

See it

a picture in your mind
a sense of direction

→

Cannot articulate

missing objects
relationships and process

→

Cannot let AI understand

AI can only guess
the result easily drifts off course

Wonderful future

Everyone create many

unprecedented worlds

